1 2 9 0

UNIVERSIDADE D
COIMBRA

dei departamento de engenharia informática

Sistemas Operativos

Licenciatura em Engenharia Informática

DEIChain

A Concurrency-Focused Blockchain Simulation

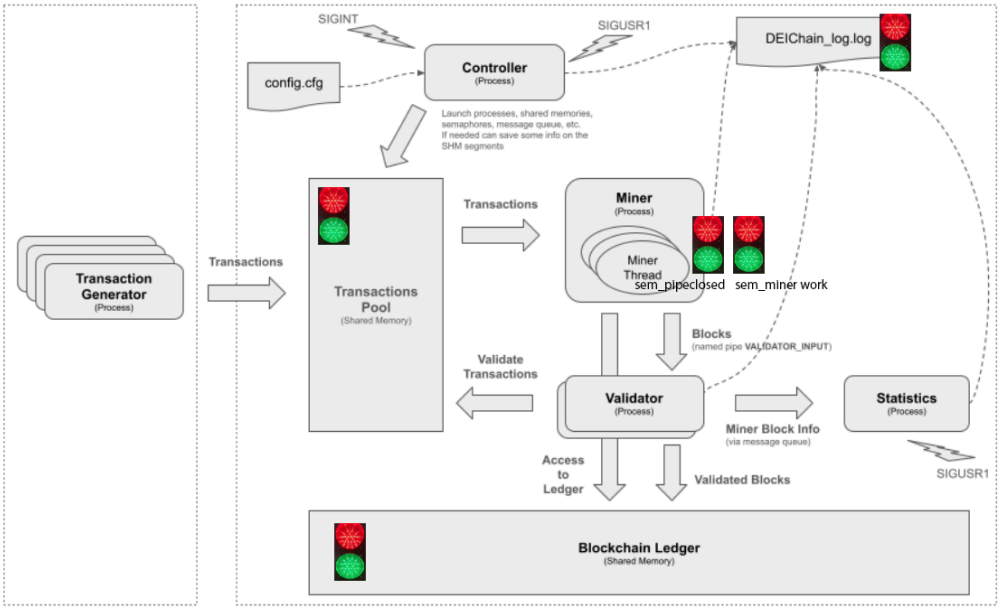
Relatório de Projeto

Diogo Rodrigues | 2022257625

Guilherme Rosmaninho | 2022257636

2025

1. Arquitetura do sistema



2. Propósito de cada estrutura utilizada

- Semaféros
    1. ***sem_transactions_pool*** - Usado para efetuar exclusão mútua da shared memory da transactions pool entre processos e threads.
    2. ***sem_ledger*** - Usado para efetuar exclusão mútua da shared memory da ledger entre processos e threads.
    3. ***sem_minerwork*** - Permite sincronizar a presença de transações para construção de novos blocos com o trabalho dos miners. Desta forma, os **miners evitam *busy waiting*** por não terem de estar constantemente a "queimar" CPU para verificar a presença de transações, usando em vez disso as estruturas do sistema operativo para garantir a eficiência destas operações.
    4. ***sem_pipeclosed*** **-** Este semáforo é usado exclusivamente para garantir o bom encerramento do pipe, de forma a que o lado de leitura (*validator*) só seja encerrado após o lado de escrita (*miner threads*) no pipe, minimizando potenciais perdas de informação em transmissão entre processos.
    5. ***sem_log_file*** - Exclusão mútua no ficheiro de log.
- Shared memories
    1. ***shm_transactionspool*** - É onde se encontram todas as transações pendentes de integração em bloco e validação.
    2. ***shm_ledger*** - Onde estão guardados os blocos que passaram validação.
- *Named pipe* para o envio de blocos que carecem de validação; comunicação *miner-validator*.
- Message queue para comunicação de dados para apresentação de estatísticas de execução da simulação.

3. Decisões gerais de desenvolvimento

- De forma a tornar mais transparente a manipulação dos dados nas *shared memories*, são criadas instâncias estruturas de dados que contêm os ponteiros para as secções relevantes nas shared memories. Como cada processo tem o seu espaço de endereçamento, é necessária uma espécie de "serialização/deserialização" dos dados a serem guardados nas shared memories, uma vez que, por exemplo, guardar a estrutura de um *TransactionBlock* iria resultar no armazenamento na shared memory de um ponteiro, algo que entre diferentes processos não tem valor e não permite o acesso à informação desejada. De forma a facilitar o desenvolvimento, recorreu-se a uma estrutura a que apelidamos de "interface", que por exemplo, aponta para cada bloco (que por sua vez aponta para os dados do bloco e a lista de transações do bloco) na shared memory e permite que se faça o acesso à ledger praticamente como uma array normal, facilitando a manipulação das memórias em todos os componentes do projeto.
- O semáforo *sem_minerwork*, previne a ocorrencia de busy waiting da seguinte forma:
    - Ao longo da sua atividade, se o *TxGen* verificar que ha transações suficientes para as *miner threads* formarem um bloco, este ira recorrer ao semáforo para desbloquear o fluxo de execução dos miners.
    - Desta forma, os *miners* só irão utilizar recursos se houver possibilidade de concretizar trabalho, prevenindo o cenário onde estes apenas estariam a utilizar recursos de forma injustificada e garantindo a eficiência por não recorrer a mecanismos como o sleep().
    - A aplicação desta estratégia apresenta, no entanto, um senão. Perante o cenário onde deixa de haver geradores ativos, a atividade dos *miners* cessaria. Para mitigar isto, os miners efetuam um *timedwait()*, fazendo por isso uma verificação periódica das transações existentes na pool, consumindo as mesmas de forma independente da existência de geradores ativos, sem que isso signifique um impacto nos recursos do sistema.
- Considerou-se que, ainda que o valor teórico para cada *stream* via pipe pudesse exceder o tamanho do buffer do pipe (que varia entre sistemas operativos), através de um aumento do número de transações por bloco no ficheiro de configurações, para o contexto do projeto se podia contar com escritas/leituras atómicas e por isso não necessitando a implementação de qualquer mecanismo para sincronizar a utilização do *pipe* e que, portanto a integridade dos dados destas transmissões é assegurada pelo sistema operativo.
- No processo *miner*, implementou-se uma thread especifica para o tratamento de sinais e gestão do término do processo. Será importante ressalvar que o recurso a *pthread_cancel()* poderia levar a um memory leak numa situação onde o cancelamento se efetua entre uma alocação e uma libertação de memória. De forma a mitigar isto, recorreu-se a uma estrutura de dados onde se encontram os ponteiros para segmentos alocados cuja libertação é gerida por *pthread_cleanup_push(cleanup_thread_resources, res)*, que permite a libertação dos recursos em uso aquando da chamada de *pthread_cancel*().
- A serialização dos dados para envio via pipe resultava na modificação do ultimo caracter/byte na transmissão de um bloco, cuja identificação só foi visível após verificar os dados transmitidos byte a byte. Este detalhe foi resolvido recorrendo à diretiva #pragma pack(push, 1) para garantir o alinhamento das estruturas na memória.